

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	V.Naga Nischay	Reg. No.:	192472255
Section / Batch:	CSE(AI)	Date:	

Code of Conduct

I V.Naga Nischay(192472255) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Nischay

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Porter Stemmer	class design, methods, encapsulation, and rule-based stemming	1

CODE:

```
def vowel(word, i):
    ch = word[i].lower()

    if ch in "aeiou":
        return True

    if ch == "y":
        if i == 0:
            return False
        return not vowel(word, i - 1)

    return False
```

```
def count(stem):
    m = 0
    flag = False

    for i in range(len(stem)):
        if vowel(stem, i):
            flag = True
        else:
            if flag:
                m += 1
            flag = False

    return m
```

```
def has_vowel(word):
    for i in range(len(word)):
        if vowel(word, i):
            return True
    return False
```

```
def double(word):
    if len(word) < 2:
        return False

    return word[-1] == word[-2] and not vowel(word, len(word) - 1)
```

```
def check(word):
    if len(word) < 3:
        return False

    if (not vowel(word, -3) and
        vowel(word, -2) and
        not vowel(word, -1)):

        return word[-1] not in "wxy"

    return False
```

```
def step1a(word):
    if word.endswith("sses"):
        return word[:-2]
    elif word.endswith("ies"):
        return word[:-2]
    elif word.endswith("ss"):
        return word
    elif word.endswith("s"):
        return word[:-1]
    return word
```

```
def step1b(word):
    if word.endswith("eed"):
        stem = word[:-1]
        if count(stem) > 0:
            return stem + "ee"

    if word.endswith("ed"):
        stem = word[:-2]
        if has_vowel(stem):
            word = stem

    elif word.endswith("ing"):
        stem = word[:-3]
        if has_vowel(stem):
            word = stem

    if word.endswith(("at", "bl", "iz")):
        word += "e"

    elif double(word) and word[-1] not in "lsz":
        word = word[:-1]

    elif count(word) == 1 and check(word):
        word += "e"

    return word
```

```
def step1c(word):
    if word.endswith("y"):
        stem = word[:-1]
        if len(stem) > 0 and vowel(stem, len(stem) - 1):
            return stem + "i"

    return word
```

```
def step2(word):
    rules = [
        ("ational", "ate"),
        ("tional", "tion"),
        ("enci", "ence"),
        ("anci", "ance"),
        ("izer", "ize"),
        ("abli", "able"),
        ("alli", "al"),
```

```

    ("entli", "ent"),
    ("eli", "e"),
    ("ousli", "ous"),
    ("ization", "ize"),
    ("ation", "ate"),
    ("ator", "ate"),
    ("alism", "al"),
    ("iveness", "ive"),
    ("fulness", "ful"),
    ("ousness", "ous"),
    ("aliti", "al"),
    ("iviti", "ive"),
    ("biliti", "ble"),
    ("logi", "log")
]

```

```

for old, new in rules:
    if word.endswith(old):
        stem = word[:-len(old)]
        if count(stem) > 0:
            return stem + new

```

```

return word

```

```

def step3(word):
    rules = [
        ("icate", "ic"),
        ("ative", ""),
        ("alize", "al"),
        ("iciti", "ic"),
        ("ical", "ic"),
        ("ful", ""),
        ("ness", "")
    ]

```

```

for old, new in rules:
    if word.endswith(old):
        stem = word[:-len(old)]
        if count(stem) > 0:
            return stem + new

```

```

return word

```

```

def step4(word):
    suffixes = [
        "al", "ance", "ence", "er", "ic", "able", "ible",
        "ant", "ement", "ment", "ent", "ion", "ou",
        "ism", "ate", "iti", "ous", "ive", "ize"
    ]

```

```

for s in suffixes:
    if word.endswith(s):
        stem = word[:-len(s)]

        if s == "ion":
            if count(stem) > 1 and (stem.endswith("s") or stem.endswith("t")):

```

```
        return stem
    else:
        if count(stem) > 1:
            return stem
```

```
    return word
```

```
def step5a(word):
    if word.endswith("e"):
        stem = word[:-1]

        if count(stem) > 1:
            return stem

        if count(stem) == 1 and not check(stem):
            return stem

    return word
```

```
def step5b(word):
    if count(word) > 1 and word.endswith("ll"):
        return word[:-1]

    return word
```

```
def stem(word):
    word = word.lower()

    word = step1a(word)
    word = step1b(word)
    word = step1c(word)
    word = step2(word)
    word = step3(word)
    word = step4(word)
    word = step5a(word)
    word = step5b(word)

    return word
```

```
word = input("Enter a word: ")
print("Stemmed Word:", stem(word))
```

Annexure A – Coding Challenge

Section 1: Object-Oriented Implementation of a Porter Stemmer

You have recently joined an **NLP startup** called **LexiMind Technologies**. The company is developing a **search engine** for a digital library containing millions of research papers. Users expect the search engine to return relevant results even when different grammatical forms of a word are used.

For example, a search for **connect** should also retrieve documents containing **connected**, **connecting**, **connection**, and **connections**. To achieve this, the development team has decided to implement the **Porter Stemming Algorithm** from scratch.

Since the algorithm consists of multiple sequential rules (Step 1a, Step 1b, Step 1c, Step 2, Step 3, Step 4, Step 5a, and Step 5b), the project manager has divided the work among team members. Each developer is responsible for implementing and testing one specific rule. At the end of the project, all modules must be integrated into a single, fully functional stemmer.

Assessment Task

Phase 1: Individual Task (Assigned Rule)

Each student will be assigned **one Porter Stemmer rule**.

Your responsibilities are to:

1. Study and understand the assigned rule.
2. Explain the linguistic purpose of the rule.
3. Describe the conditions under which the rule should be applied.
4. Implement the rule in Python.
5. Prepare at least **10 test words** demonstrating:
 - Words that satisfy the rule.
 - Words that should remain unchanged.
6. Document the input, expected output, and actual output.
7. Explain any limitations or special cases encountered.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1	20	25	20	20	10	95	95

Faculty In-charge	Course Coordinator	Program Director
Dr.T.Kumaragurubaran		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____